K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Name of Student : Harshal Nitin Mehta
Roll No : B - 63
Batch : SB4

EXPERIMENT 3

Aim:

To implement a Double-ended Queue (Deque) using an array and demonstrate Input Restricted Deque and Output Restricted Deque operations.

Objective:

- To implement a double-ended queue using an array.
- To perform insertion and deletion operations from both ends of the deque.
- To implement Input Restricted Deque.
- To implement Output Restricted Deque.
- To understand overflow and underflow conditions.

Software Used:

- Turbo C / Turbo C++
- DOSBox

Theory:

A Double-ended Queue (Deque) is a linear data structure in which insertion and deletion can be performed from both ends.

There are two types of **restricted deque:**

1. **Input Restricted Deque:**
   Insertion is allowed only from one end, while deletion is allowed from both ends.

2. **Output Restricted Deque:**
   Deletion is allowed only from one end, while insertion is allowed from both ends.

In this experiment, an array is used to implement the deque. The variables left and right are used to keep track of the two ends of the deque.

Program :

```
#include <stdio.h>
#include <conio.h>

#define MAX 10

int deque[MAX];
int left = -1, right = -1;

void insert_left()
{
   int x;

   if(left == (right + 1) % MAX)
   {
      printf("\nOVERFLOW");
      return;
   }

   printf("\nEnter value: ");
```

```
    scanf("%d", &x);

    if(left == -1)
        left = right = 0;
    else
    {
        left--;
        if(left < 0)
            left = MAX - 1;
    }

    deque[left] = x;
}

void insert_right()
{
    int x;

    if(left == (right + 1) % MAX)
    {
        printf("\nOVERFLOW");
        return;
    }

    printf("\nEnter value: ");
    scanf("%d", &x);

    if(left == -1)
        left = right = 0;
    else
    {
        right++;
        if(right == MAX)
            right = 0;
    }

    deque[right] = x;
}

void delete_left()
```

```
{
    if(left == -1)
    {
        printf("\nUNDERFLOW");
        return;
    }

    printf("\nDeleted: %d", deque[left]);

    if(left == right)
        left = right = -1;
    else
    {
        left++;
        if(left == MAX)
            left = 0;
    }
}

void delete_right()
{
    if(left == -1)
    {
        printf("\nUNDERFLOW");
        return;
    }

    printf("\nDeleted: %d", deque[right]);

    if(left == right)
        left = right = -1;
    else
    {
        right--;
        if(right < 0)
            right = MAX - 1;
    }
}

void display()
```

```
{
    int i;

    if(left == -1)
    {
        printf("\nQUEUE IS EMPTY");
        return;
    }

    printf("\nElements: ");

    i = left;

    while(i != right)
    {
        printf("%d ", deque[i]);
        i++;
        if(i == MAX)
            i = 0;
    }

    printf("%d", deque[right]);
}

int main()
{
    int choice, option;

    clrscr();

    printf("\n1. Input Restricted Deque");
    printf("\n2. Output Restricted Deque");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    do
    {
        printf("\n\n1. Insert Right");

        if(choice == 2)
```

```
            printf("\n2. Insert Left");

        printf("\n3. Delete Left");

        if(choice == 1)
            printf("\n4. Delete Right");

        printf("\n5. Display");
        printf("\n6. Exit");
        printf("\nEnter option: ");
        scanf("%d", &option);

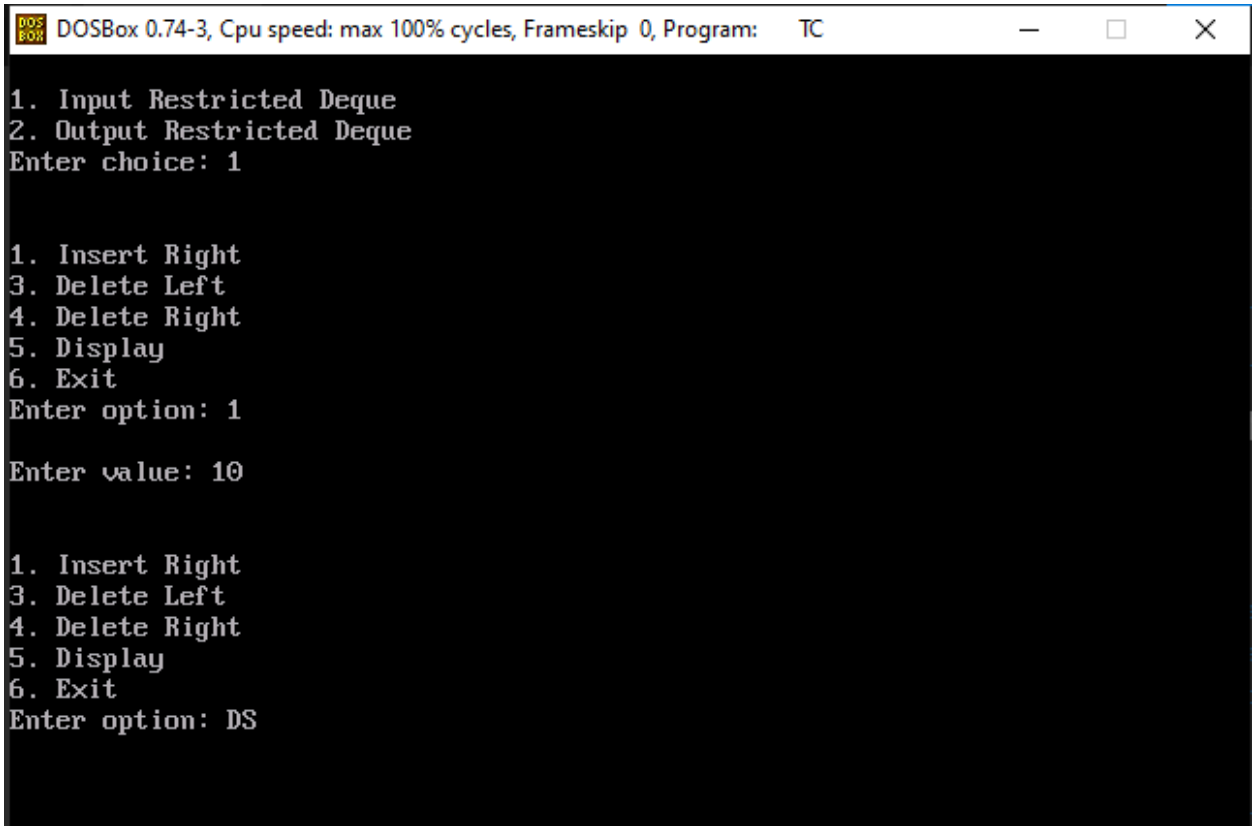
        if(option == 1)
            insert_right();
        else if(option == 2 && choice == 2)
            insert_left();
        else if(option == 3)
            delete_left();
        else if(option == 4 && choice == 1)
            delete_right();
        else if(option == 5)
            display();
        else if(option != 6)
            printf("\nInvalid option.");

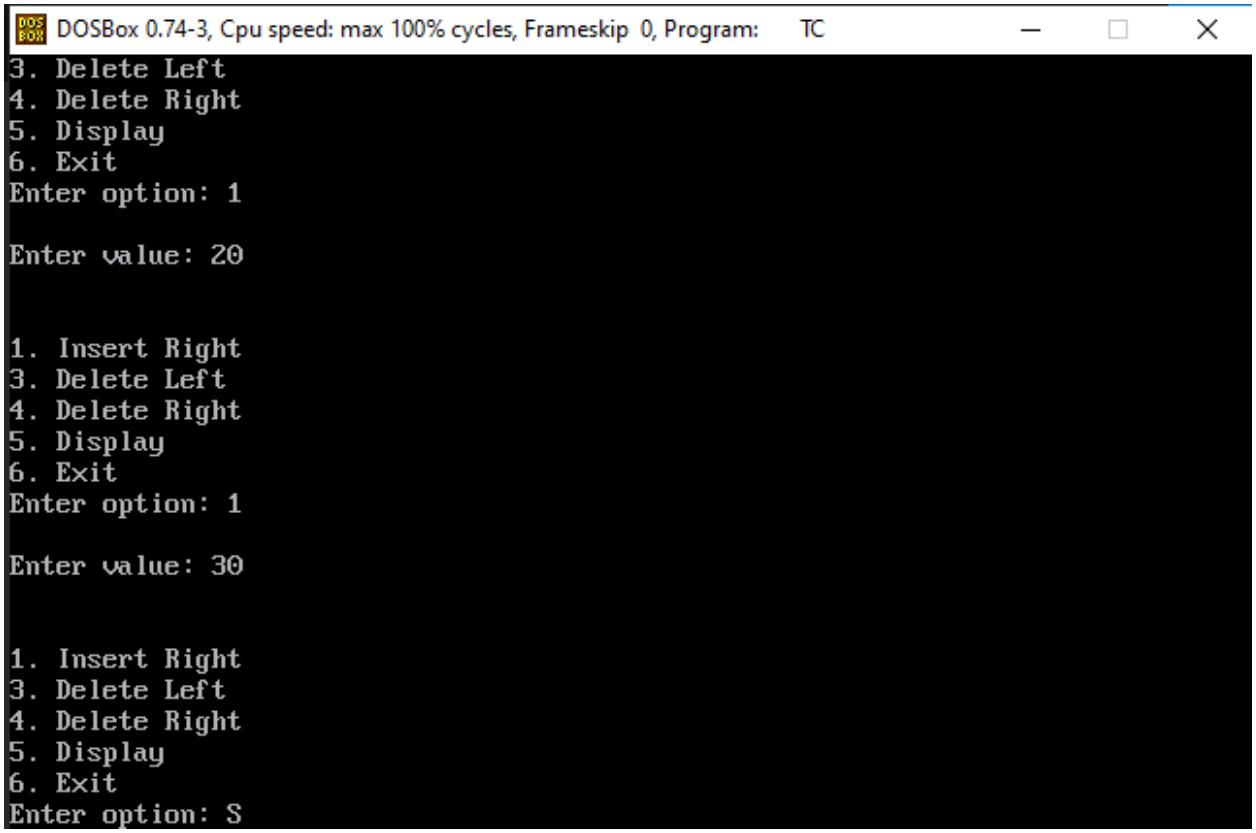
    } while(option != 6);

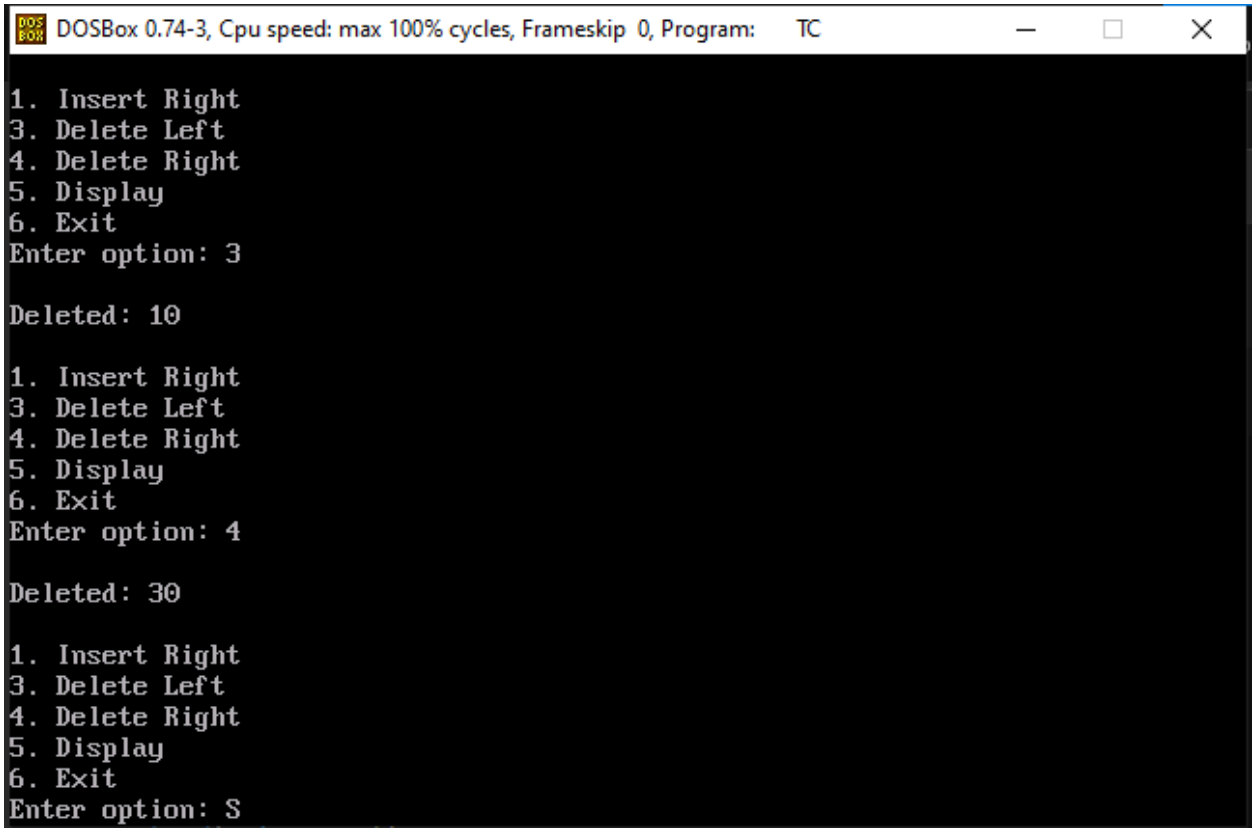
    return 0;
}
```

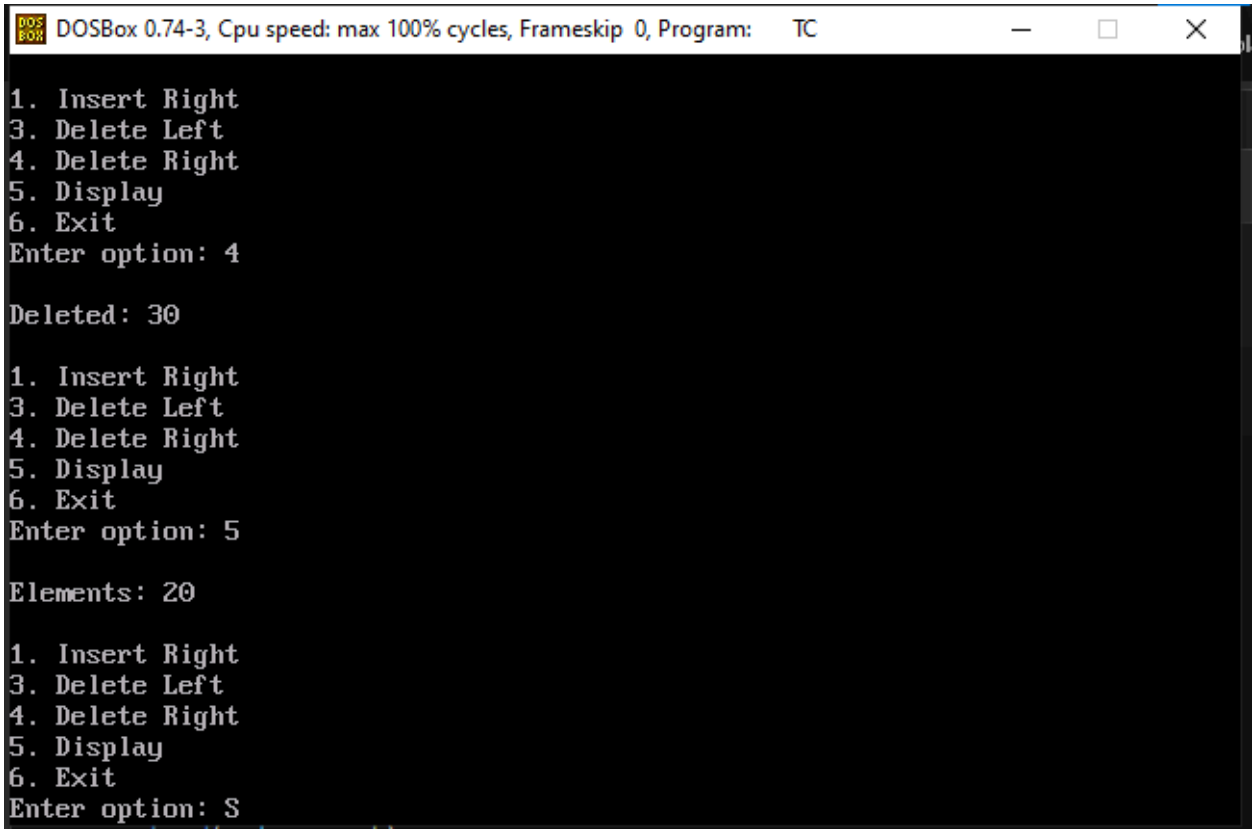
Output :

Input Restricted Queue

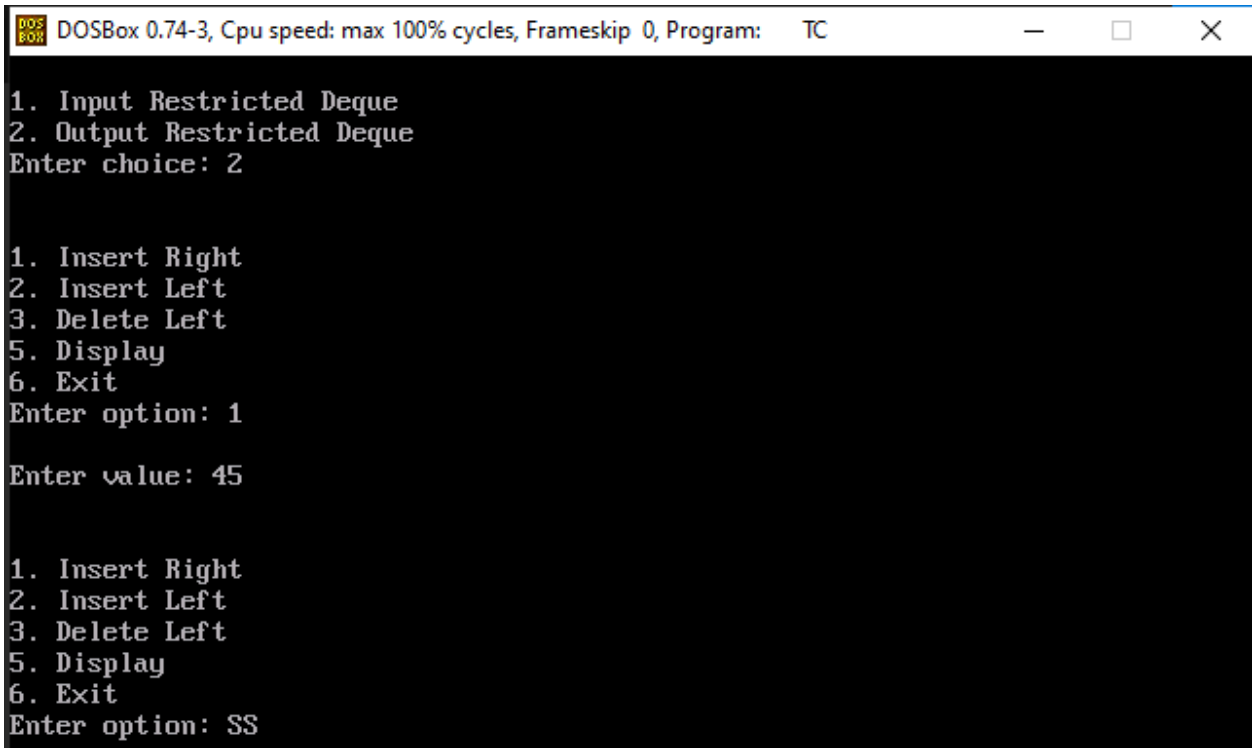

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1. Input Restricted Deque
2. Output Restricted Deque
Enter choice: 1
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 1
Enter value: 10
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: DS

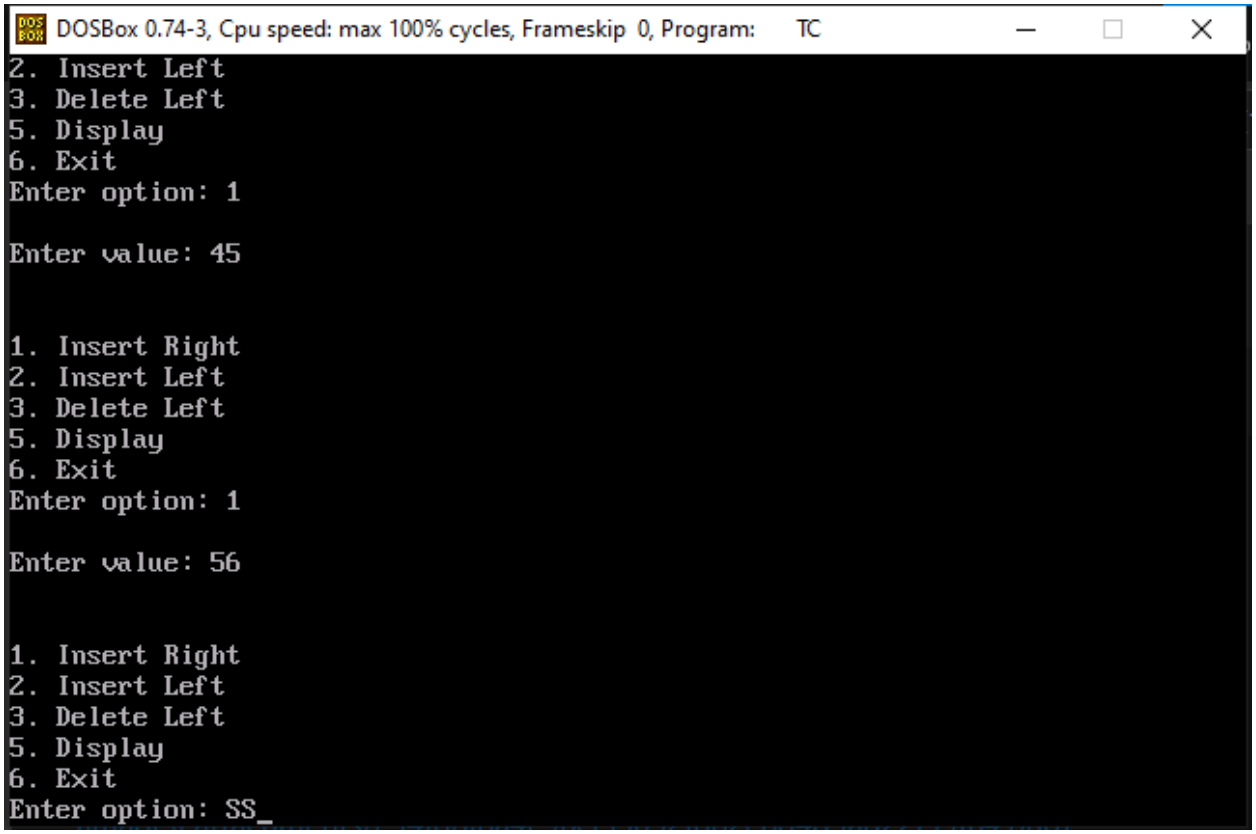

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 1
Enter value: 20
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 1
Enter value: 30
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: S

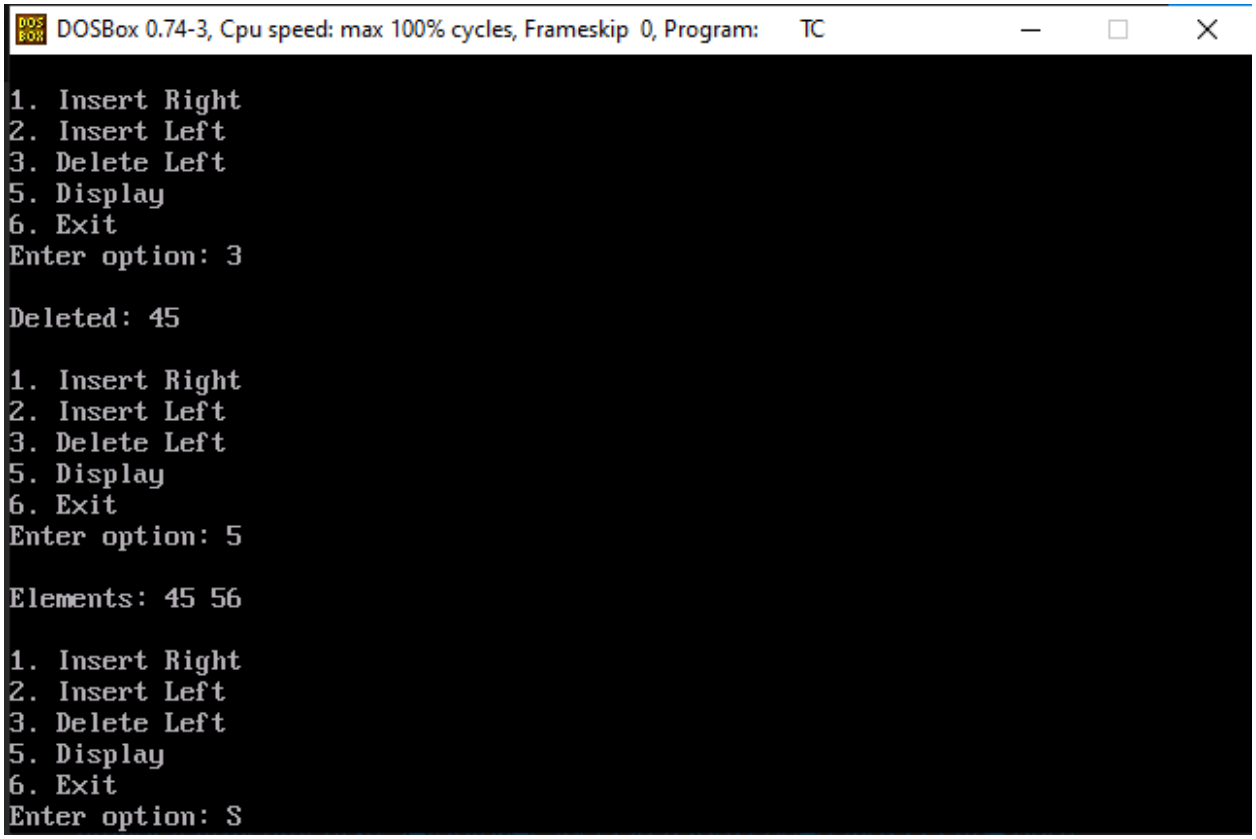

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 3
Deleted: 10
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 4
Deleted: 30
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: S

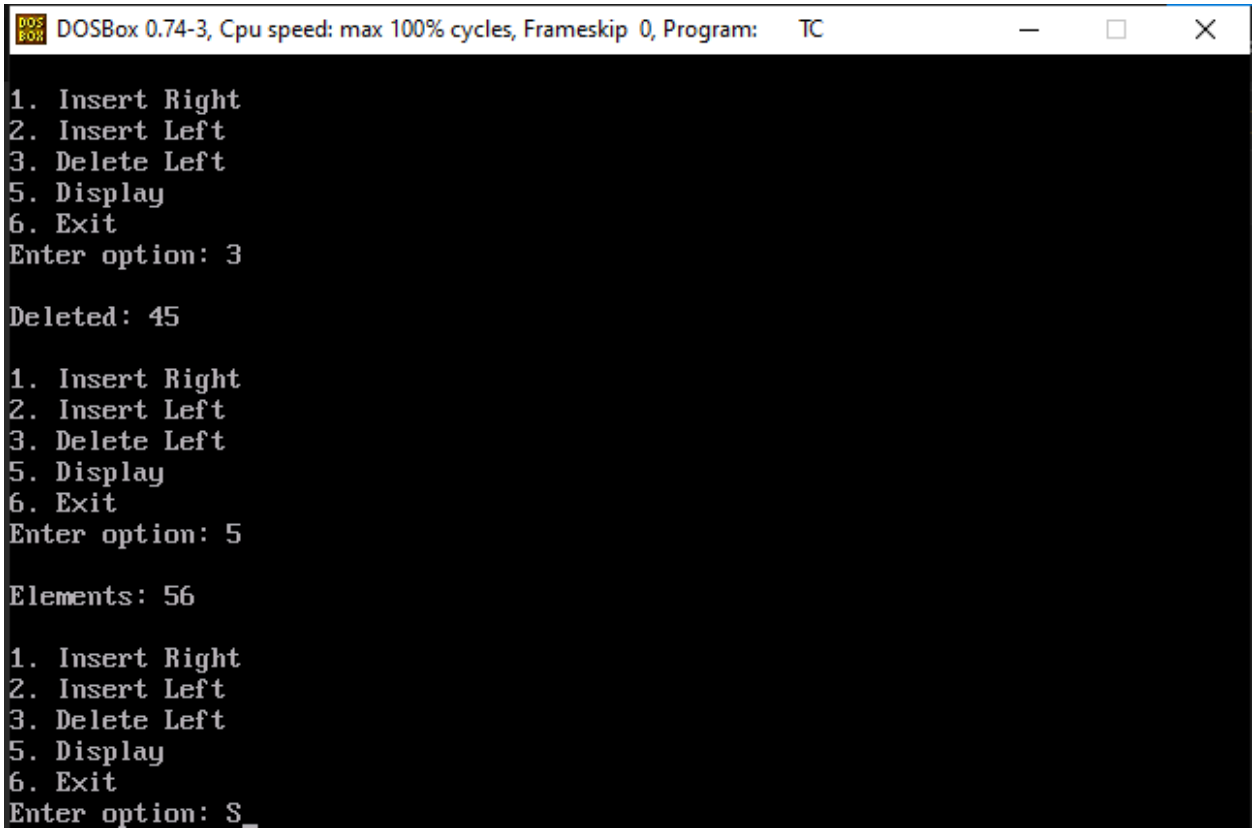

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 4
Deleted: 30
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: 5
Elements: 20
1. Insert Right
3. Delete Left
4. Delete Right
5. Display
6. Exit
Enter option: S

Output Restricted Queue :




DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: 1
Enter value: 45
1. Insert Right
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: 1
Enter value: 56
1. Insert Right
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: SS_


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
1. Insert Right
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: 3
Deleted: 45
1. Insert Right
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: 5
Elements: 45 56
1. Insert Right
2. Insert Left
3. Delete Left
5. Display
6. Exit
Enter option: S



Outcome :

The program successfully implements a double-ended queue using an array and demonstrates Input Restricted Deque and Output Restricted Deque operations.

Conclusion :

The experiment demonstrates the implementation of a Deque using an array and the different insertion and deletion restrictions of Input Restricted and Output Restricted Deques.
